NOVA
MASTER TECHNICAL MANUAL

v2.0 Evolution Release

Architecture | Intelligence | Automation

Table of Contents

NOVA SYSTEM AI - COMPLETE TECHNICAL DOCUMENTATION

PART I: PROJECT OVERVIEW

Chapter 1: Introduction to NOVA

1.1 What is NOVA?

NOVA (Natural Operational Voice Assistant) is an advanced AI-powered desktop assistant designed for Windows systems. It combines multiple AI technologies including:

- Local Language Models (via Ollama)
- Cloud AI Integration (Groq API)
- Custom Neural Intent Engine (from-scratch Transformer)
- Voice Control (Speech Recognition + TTS)
- System Control Automation
- Mobile Remote Access (BLE/HTTP Bridge)
- MCP Agent (Code Generation & Execution)

1.2 Project Architecture (v2.0 Organized)

```
 Nova-System-AI/
-- data/                 # Persistent Storage (SQLite, Habits, Memory)
-- interface/            # Frontend (HUD, Style, Assets)
-- nova_system/          # Core Logic (Brains, Automation, Neural Engine)
-- agent/                # Autonomous Agent Loop & Tools
-- docs/                 # Documentation & PDF Generators
-- workspace/            # Sandboxed Execution Environment
+-- nova_cli.py           # Master Entry Point
```

1.2.1 Directory Purpose

Directory	Content Type	Role	Implementation
`/data`	Binary/JSON	Persistent storage for long-term intelligence	SQLite, user_memory.json
`/interface`	HTML/CSS/JS	Graphical HUD and mobile remote assets	PyQt6, Jinja2 Templates
`/nova_system`	Python	Core AI orchestration and provider bridge	MultiModelBrain, UCE
`/agent`	Python / tools	Decision-making logic and local tool execution	ReAct Loop, Subprocess
`/workspace`	Temporary	Sandbox for code generation and test scripts	Read/Write/Execute
`/docs`	PDF / MD	Technical specifications and manual outputs	fpdf2, Markdown
`/agent/tools`	Specialized	Extensible modules for system mastery	Git, WebSearch, SystemMonitor

1.2.2 Intelligence Node Infrastructure

The system operates across three distinct intelligence nodes to ensure 99.9% uptime regardless of internet connectivity:

1. Local Node (Privacy): Powered by Ollama running Llama-3 (8B) or Mistral. Handles sensitive local file analysis.

2. Speed Node (Latency): Powered by Groq Cloud. Provides near-instant (0.3s) response times for voice interaction.

3. Reasoning Node (Logic): Powered by Gemini 1.5 Pro. Used for processing large codebases and complex mathematical planning.

1.2.3 Communication Protocols

NOVA maintains a persistent sync across the following protocols:

- RFCOMM (Bluetooth): Serial control for legacy hardware remotes.

-

BLE (WebBridge): High-speed browser interface for mobile dashboards.

- WebSocket (HUD): Internal IPC between the Python core and the PyQt6 GUI.
- StdIn/StdOut (CLI): Fallback terminal interface for advanced developers.

1.3 Key Features

Feature	Description
Multi-Model Brain	Dynamic failover between Groq, Gemini, and Ollama
Autonomous Agent	ReAct loop for step-by-step task completion
Neural Evolution	Self-optimization of intent classification
Visual HUD	PyQt6-based graphical interface with vision analyzer
Voice Interface	Female voice (Zira) with robust STT/TTS
Self-Programming	Ability to read and modify its own source code
Project Manager	Auto-documentation and code review capabilities

---

PART II: NEURAL INTENT ENGINE (NIE)

Chapter 2: Transformer Architecture

2.1 Model Overview

The Neural Intent Engine uses a TinyTransformerClassifier - a from-scratch implementation using only NumPy.
Architecture Specifications:

- Embedding Dimension: 64
- Number of Layers: 2
- Attention Heads: 2
- Vocabulary Size: 500-1000
- Classification Classes: 5

2.2 Mathematical Foundation

2.2.1 Token Embedding

The input text is converted to token IDs and embedded:

```
h = W_tok[x] + W_pos[:seq_len]
```

Where:

- `W_tok` in R^(vocab_size x dim) is the token embedding matrix
- `W_pos` in R^(max_seq x dim) is the positional embedding
- `x` is the input token sequence

2.2.2 Self-Attention Mechanism

For each layer i, compute Query, Key, Value:

```
Q = h x W_q
K = h x W_k
V = h x W_v
```

Scaled Dot-Product Attention:

```
Attention(Q, K, V) = softmax(QK.T / sqrtd_k) x V
```

Mathematical derivation:

1. Score Calculation:

```
scores = Q x K.T
```

Shape: (seq_len x seq_len)

2. Scaling:

```
scaled_scores = scores / sqrtdim
```

This prevents gradient vanishing in softmax.

3. Softmax Normalization:

```
attention_weights = exp(scaled_scores) / Sumexp(scaled_scores)
```

4. Weighted Value Sum:

```
   attention_output = attention_weights  x  V
```

5. Output Projection with Residual:

```
   h = h + (attention_output  x  W_o)
```

2.2.3 Feed-Forward Network

Each layer includes an FFN:

```
FFN(h) = ReLU(h  x  W_1)  x  W_2
```

Where:

- `W_1` in R^(dim x 4dim) - expansion layer
- `W_2` in R^(4dim x dim) - compression layer
- ReLU(x) = max(0, x)

With residual connection:

```
h = h + FFN(h)
```

2.2.4 Classification Head

Mean Pooling:

```
pooled = (1/seq_len)  x  Sum h_i
```

Final Classification:

```
 logits = pooled  x  W_final
probs = softmax(logits)
predicted_class = argmax(probs)
```

2.3 Implementation Code

```
class TinyTransformerClassifier:
   def __init__(self, vocab_size, num_classes=5, dim=64, layers=2, heads=2):
       self.vocab_size = vocab_size
       self.num_classes = num_classes
       self.dim = dim
       self.layers = layers
       self.heads = heads
       self.params = self._init_weights()

   def _init_weights(self):
       """Xavier initialization for stable training."""
       p = {}
       p['w_tok'] = np.random.randn(self.vocab_size, self.dim) / np.sqrt(self.dim)
       p['w_pos'] = np.random.randn(32, self.dim) / np.sqrt(self.dim)

       for i in range(self.layers):
           p[f'l{i}_wq'] = np.random.randn(self.dim, self.dim) / np.sqrt(self.dim)
           p[f'l{i}_wk'] = np.random.randn(self.dim, self.dim) / np.sqrt(self.dim)
           p[f'l{i}_wv'] = np.random.randn(self.dim, self.dim) / np.sqrt(self.dim)
           p[f'l{i}_wo'] = np.random.randn(self.dim, self.dim) / np.sqrt(self.dim)
           p[f'l{i}_w1'] = np.random.randn(self.dim, self.dim  4) / np.sqrt(self.dim)
           p[f'l{i}_w2'] = np.random.randn(self.dim  4, self.dim) / np.sqrt(self.dim  4)

       p['w_final'] = np.random.randn(self.dim, self.num_classes) / np.sqrt(self.dim)
       return p

   def forward(self, x):
       seq_len = len(x)
       h = self.params['w_tok'][x] + self.params['w_pos'][:seq_len]

       for i in range(self.layers):
           q = h @ self.params[f'l{i}_wq']
           k = h @ self.params[f'l{i}_wk']
           v = h @ self.params[f'l{i}_wv']

           attn_scores = (q @ k.T) / np.sqrt(self.dim)
           attn_weights = self._softmax(attn_scores)
           attn_out = attn_weights @ v
           h = h + attn_out @ self.params[f'l{i}_wo']

           ff = self._relu(h @ self.params[f'l{i}_w1']) @ self.params[f'l{i}_w2']
           h = h + ff

       pooled = np.mean(h, axis=0)
       logits = pooled @ self.params['w_final']
       probs = self._softmax(logits)
       return probs

   def _softmax(self, x):
       e_x = np.exp(x - np.max(x, axis=-1, keepdims=True))
       return e_x / e_x.sum(axis=-1, keepdims=True)

   def _relu(self, x):
       return np.maximum(0, x)
```

Chapter 3: Training Algorithm

3.1 Training Strategy

NOVA uses Prototype Alignment Training - a simplified gradient-based approach.

3.2 Loss Function

Cross-Entropy Loss:

```
L = -Sum y_i  x  log(p_i)
```

For single-class prediction:

```
L = -log(p_target)
```

```
Gradient of Softmax + Cross-Entropy:
```

```
dL/dlogits = probs - one_hot(target)
```

3.3 Gradient Descent with Momentum

Update Rule:

```
 velocity = momentum  x  velocity - learning_rate  x  gradient
params = params + velocity
```

Parameters:

- Learning Rate (lr): 0.01
- Momentum: 0.9
- Epochs: 300

3.4 Gradient Calculation

Classification Head Gradient:

```
 grad_logits = probs.copy()
grad_logits[target] -= 1  # = probs - one_hot

g_w_final = np.outer(pooled, grad_logits)
g_w_final = np.clip(g_w_final, -1.0, 1.0)  # Gradient clipping
```

Token Embedding Alignment:

```
 target_vec = model.params['w_final'][:, target]
for token_id in input_ids:
    g_tok = (model.params['w_tok'][token_id] - target_vec)
    g_tok = np.clip(g_tok, -1.0, 1.0)
    model.params['w_tok'][token_id] -= lr  0.1  g_tok
```

3.5 Dataset Generation

Intent Categories:

ID	Intent	Examples
0	LOCK_SYSTEM	"lock computer", "secure laptop"
1	VOLUME_UP	"louder", "increase volume"

2	VOLUME_DOWN	"quieter", "decrease volume"
3	SYSTEM_STATUS	"how are you", "battery status"
4	UNKNOWN	"hello", "weather"

Data Augmentation:

```
for example in examples:
   dataset.append({"text": example, "label": intent_id})
   dataset.append({"text": example + "!", "label": intent_id})
   dataset.append({"text": "can you " + example, "label": intent_id})
   dataset.append({"text": "please " + example, "label": intent_id})
```

Chapter 4: Tokenization

4.1 Word-Level Tokenizer

```
class SimpleWordTokenizer:
   def __init__(self):
       self.word_to_id = {"[PAD]": 0, "[UNK]": 1}
       self.id_to_word = {0: "[PAD]", 1: "[UNK]"}
       self.vocab_size = 2

   def encode(self, text, max_len=12):
       text = text.lower().strip()
       words = re.findall(r'\w+', text)
       ids = []
       for w in words[:max_len]:
           if w not in self.word_to_id:
               self._add_word(w)
           ids.append(self.word_to_id[w])
       padding = [0]  (max_len - len(ids))
       return ids + padding
```

4.2 Vocabulary Building

- Dynamic vocabulary growth during encoding
- Pre-populated with common command words
- Stored as JSON for persistence

PART III: PERMISSION GATE SYSTEM

Chapter 5: Safety Layer

5.1 Human-in-the-Loop Design

The PermissionGate separates "thinking" from "doing":

```
class PermissionGate:
   @staticmethod
   def ask_permission(intent_name, confidence):
       print(f"Detected Intent: {intent_name}")
       print(f"Confidence: {confidence100:.2f}%")
       choice = input(f"Confirm execution? (y/n): ")
       return choice == 'y'

   @staticmethod
   def execute_intent(intent_id):
       if intent_id == 0:  # LOCK_SYSTEM
           subprocess.run("rundll32.exe user32.dll,LockWorkStation")
       elif intent_id == 1:  # VOLUME_UP
           # PowerShell volume control
       elif intent_id == 2:  # VOLUME_DOWN
           # PowerShell volume control
       elif intent_id == 3:  # SYSTEM_STATUS
           print(f"CPU: {psutil.cpu_percent()}%")
```

5.2 Confidence Threshold

Only execute when confidence 75%:

```
if confidence >= 0.75 and intent_name != "UNKNOWN":
   if PermissionGate.ask_permission(intent_name, confidence):
       PermissionGate.execute_intent(intent_id)
```

PART IV: MCP AGENT SYSTEM

Chapter 6: Model Context Protocol

6.1 Tool Architecture

The MCP Agent provides code generation and execution:

```
class MCPTool:
   name: str
   description: str

   def execute(self, kwargs) -> Dict[str, Any]:
       raise NotImplementedError
```

6.2 Available Tools

Tool	Description
create_python_file	Create Python files
execute_python_file	Run Python files
execute_python_code	Run inline code
read_file	Read file contents
list_files	List directory
search_files	Find files
file_tree	Directory tree
fetch_file	Trie-based search

6.3 Security Checks

Blocked Patterns:

```
BLOCKED_PATTERNS = [
   r"os\.system\s\(",
   r"subprocess\.call\s\(",
   r"exec\s\(",
   r"eval\s\(",
   r"rm\s+-rf",
   r"format\s+[a-zA-Z]:",
]
```

6.4 Trie Data Structure (Advanced Search)

Prefix Tree for Fast File Search:

```
 class TrieNode:
    def __init__(self):
        self.children: Dict[str, 'TrieNode'] = {}
        self.is_end_of_word = False
        self.full_path = ""

class FileTrieIndexerTool:
    def _insert(self, filename: str, full_path: str):
        node = self.root
        for char in filename:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.is_end_of_word = True
        node.full_path = full_path

    def _search(self, prefix: str) -> List[str]:
        node = self.root
        for char in prefix:
            if char not in node.children:
                return []
            node = node.children[char]
        results = []
        self._collect_all(node, results)
        return results
```

Complexity Analysis:

- Indexing (Pre-computation): O(N L) where N is the number of files and L is the average path length.
- Search Latency: O(M + K) where M is the query length and K is the number of results found.
- Memory Overhead: Significant, as each character creates a node, but mitigated by using Python's `__slots__` if enabled.

PART V: UNIFIED CONTROL ENGINE (UCE)

Chapter 7: System Mastery & Orchestration

7.0 Overview of UCE Logic

The UCE is the automation core. It translates the high-level intent detected by the NIE into a sequence of low-level OS calls.

7.1 System Status Monitoring (Deep Telemetry)

Nova samples 15+ sensors every 500ms to maintain a "System Awareness" state:

```
class SystemControl:
   @staticmethod
   def get_system_status():
       return {
           "cpu": {
               "usage": psutil.cpu_percent(interval=None),
               "freq": psutil.cpu_freq().current,
               "cores": psutil.cpu_count()
           },
           "memory": {
               "total": psutil.virtual_memory().total,
               "available": psutil.virtual_memory().available,
               "percent": psutil.virtual_memory().percent
           },
           "battery": {
               "percent": psutil.sensors_battery().percent if psutil.sensors_battery() else 100,
               "power_plugged": psutil.sensors_battery().power_plugged if psutil.sensors_battery() else
           },
           "thermal": psutil.sensors_temperatures() if hasattr(psutil, "sensors_temperatures") else "N/A
       }
```

7.2 Application Lifecycle Management

Nova maintains a fuzzy-mapped cache of all executables in `C:\ProgramData\Microsoft\Windows\Start Menu\Programs`.

Fuzzy Matching Algorithm (Implementation):

```
def find_app(self, query):
   best_match = None
   best_ratio = 0
   for name, path in self.apps.items():
       # Using SequenceMatcher to allow for typos like 'notpad' vs 'notepad'
       ratio = SequenceMatcher(None, query.lower(), name.lower()).ratio()
       if ratio > best_ratio and ratio > 0.6:
           best_match = name
           best_ratio = ratio
   return (best_match, self.apps[best_match]) if best_match else (None, None)
```

7.3 Volume & Audio Orchestration

Using Windows COM API via PowerShell integration:

```
def set_volume(level: int):
   # Normalize level to 0-100 and send VK_VOLUME_UP/DOWN keys via WScript
   subprocess.run(f'''powershell "$wsh = New-Object -ComObject WScript.Shell;
   for($i=0; $i -lt {level//4}; $i++) {{ $wsh.SendKeys([char]175) }}"''')
```

PART VI: VOICE CONTROL

Chapter 8: Speech Processing

8.1 Text-to-Speech

```
 import pyttsx3
TTS_ENGINE = pyttsx3.init()
voices = TTS_ENGINE.getProperty('voices')
TTS_ENGINE.setProperty('voice', voices[1].id)  # Female voice
TTS_ENGINE.setProperty('rate', 175)

def speak(text):
    TTS_ENGINE.say(text)
    TTS_ENGINE.runAndWait()
```

8.2 Speech Recognition

```
 import speech_recognition as sr
recognizer = sr.Recognizer()

def listen(timeout=5):
    with sr.Microphone() as source:
        recognizer.dynamic_energy_threshold = True
        recognizer.energy_threshold = 300
        recognizer.adjust_for_ambient_noise(source, duration=1.0)
        audio = recognizer.listen(source, timeout=timeout)
    return recognizer.recognize_google(audio, language='en-US')
```

PART VII: MOBILE INTEGRATION

Chapter 9: BLE Bridge Server

9.1 HTTP-based Mobile Interface

```
class BleServer:
   def __init__(self):
       self.port = 8888
       self.device_name = "Nova-BLE"

   def start(self):
       handler = self._create_handler()
       self.server = HTTPServer(('0.0.0.0', self.port), handler)
       threading.Thread(target=self._run_server, daemon=True).start()
```

9.2 Mobile UI Features

- Real-time system diagnostics
- Volume/Brightness control
- App launcher
- AI Chat interface
- PIN-based unlock

PART VIII: MODULES REFERENCE

Chapter 10: Python Dependencies

10.1 Core Dependencies

Module	Purpose
numpy	Neural network math
psutil	System monitoring
rich	Terminal UI
pyttsx3	Text-to-speech
speech_recognition	Voice input
requests	HTTP requests
pyserial	Bluetooth

10.2 Optional Dependencies

Module	Purpose
groq	Groq AI API
google-generativeai	Gemini API
huggingface-hub	HF models
pywin32	Windows API

PART IX: HOW THE MODEL RUNS

Chapter 11: Execution Flow

11.1 Startup Sequence

```
 1. Load environment variables (.env)
2. Initialize TTS engine
3. Initialize App Finder (scan installed apps)
4. Check Ollama availability
5. Load Neural Intent Engine weights
6. Start main chat loop
```

11.2 Message Processing Pipeline

```
 User Input
    v
+-----------------------------+
| 1. Check for slash commands |
|    (/help, /status, etc.)   |
+-----------------------------+
    v
+-----------------------------+
| 2. Parse action commands    |
|    (open X, close X, etc.)  |
+-----------------------------+
    v
+-----------------------------+
| 3. Try Neural Intent Engine |
|    (if confidence > 75%)    |
+-----------------------------+
    v
+-----------------------------+
| 4. Send to LLM              |
|    (Ollama/Groq/Custom)     |
+-----------------------------+
    v
+-----------------------------+
| 5. Parse LLM response       |
|    (extract tool calls)     |
+-----------------------------+
    v
+-----------------------------+
| 6. Execute tools            |
|    (with safety checks)     |
+-----------------------------+
    v
Output to User
```

11.3 LLM Integration

Ollama (Local):

```
 response = requests.post(
    f"{OLLAMA_URL}/api/generate",
    json={"model": model_name, "prompt": prompt, "stream": False}
)
```

Groq (Cloud):

```
 client = Groq(api_key=GROQ_API_KEY)
response = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=messages,
    max_tokens=500
)
```

PART X: MATHEMATICAL APPENDIX

Appendix A: Softmax Function

Definition:

```
softmax(x_i) = exp(x_i) / Sum_j exp(x_j)
```

Numerical Stability:

```
def softmax(x):
   e_x = np.exp(x - np.max(x))  # Subtract max for stability
   return e_x / e_x.sum()
```

Derivative:

```
dsoftmax(x_i)/dx_j = softmax(x_i)  x  (delta_ij - softmax(x_j))
```

Appendix B: Xavier Initialization

Formula:

```
W ~ N(0, 1/sqrtn_in)
```

Where n_in is the number of input neurons.
Purpose: Maintains variance across layers, preventing vanishing/exploding gradients.

Appendix C: Cross-Entropy Loss

Formula:

```
L = -Sum y_i  x  log(p_i)
```

Gradient:

```
dL/dp_i = -y_i/p_i
```

Combined with Softmax:

```
dL/dz_i = p_i - y_i
```

Appendix D: Attention Mathematics

Scaled Dot-Product:

```
Attention(Q,K,V) = softmax(QK.T/sqrtd_k)V
```

Why scale by sqrtd_k?

- For large d_k, dot products grow large
- Large values push softmax to extreme values
- This causes vanishing gradients
- Scaling maintains gradient flow

PART XI: INSTALLATION & USAGE

Chapter 12: Getting Started

12.1 Installation

```
 git clone https://github.com/YOUR_USERNAME/Nova-System-AI.git
cd Nova-System-AI
pip install -r requirements.txt
```

12.2 Running NOVA

```
python nova_cli.py
```

12.3 Commands

Command	Description
/help	Show all commands
/status	System status
/model	Change AI model
/voice	Start voice mode
/web	Start mobile server
/agent	Start MCP agent
/exit	Exit NOVA

PART XI: PRESENTATION HIGHLIGHTS

Chapter 11: Speaker Talking Points

11.1 The "Why" behind NOVA

- Independence: Most AI assistants are fragile wrappers. Nova is a sturdy engine with its own neural intent classifier.
- Privacy: Local data storage and Ollama support mean Nova can function entirely offline for sensitive tasks.
- Self-Programming: Nova is the only assistant that can literally "rewrite itself" to adapt to new system requirements.

11.2 Key Technical Achievements

- Neural Layer: Implementing a Transformer in pure NumPy with backpropagation and momentum.
- Agentic Layer: A ReAct-based agent that doesn't just chat, but plans and executes using a suite of 20+ specialized tools.
- Control Layer: Seamless integration between Python and Windows APIs for full system mastery.

CONCLUSION

NOVA System AI represents the next step in personalized autonomous computing. It is not just a chatbot; it is a Neural OS Companion.

Document Version: 2.0

Generated: January 2026

Project: Nova-System-AI